

# 1. Visión general del sistema

---

**Nombre:** Mi Colegio Favorito

**Naturaleza:** sistema integral de gestión escolar **multi-centro**.

**Propósito:** matrícula, notas y kardex, asistencia, disciplina, nómina docente, caja (facturación y recibos), certificados con flujo de aprobación/cobro/entrega, comunicaciones (correo/WhatsApp), reportes académicos y dashboard administrativo.

Cada centro educativo "contrata" módulos de forma independiente. Los flujos de los módulos inactivos se desactivan solos (gate de URLs, portales sin cobros, certificados gratuitos auto-aprobados, deuda neutral).

## 1.1 Stack tecnológico

| Capa          | Tecnología                                                    |
|---------------|---------------------------------------------------------------|
| Lenguaje      | Python 3.14                                                   |
| Framework     | Django 6                                                      |
| Base de datos | PostgreSQL (producción) / SQLite (desarrollo, con WAL)        |
| Estáticos     | Whitenoise                                                    |
| Servidor      | Gunicorn                                                      |
| Caché         | Redis (opcional; sin Redis usa LocMemCache)                   |
| 2FA           | TOTP (Google Authenticator, Authy, etc.)                      |
| Seguridad     | CSP, cifrado Fernet de datos sensibles, middleware de defensa |
| Correo        | SMTP / SendGrid / cualquier API (Anymail)                     |

## 1.2 Aplicaciones (apps) del sistema

---

| App              | Responsabilidad                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| usuarios         | Login/logout, 2FA, creación de miembros, roles de usuario                                                                                  |
| core             | Centros educativos, años escolares, configuración por centro, permisos de página, tema/logo, middleware y tareas                           |
| estudiantes      | Estudiantes, matrícula/inscripción, kardex, constancias, disciplina, solicitudes de certificados, historial clínico, portales estudiante   |
| docentes         | Docentes, panel del docente, calificaciones por asignación                                                                                 |
| tutores          | Tutores/padres, portal del tutor, solicitudes, historial clínico                                                                           |
| academico        | Grados, secciones, asignaturas, períodos, competencias, calificaciones, carga académica                                                    |
| administracion   | Dashboard administrativo, seguimiento académico, boletines, personal, mantenimiento                                                        |
| reportes         | Hub de reportes (disponibles, consultas, métricas), reportes de asistencia, calificaciones, carga académica, boletas/planillas imprimibles |
| promociones      | Dashboard de cierre y promociones (auto-promoción, recuperación)                                                                           |
| asistencia       | Asistencia de estudiantes, resumen, días no docencia, QR y biométrico                                                                      |
| caja             | Caja, pagos, egresos, cuentas por cobrar, conceptos, reporte diario                                                                        |
| facturacion      | Facturación, facturas, secuencias NCF, tipos de comprobante                                                                                |
| nomina           | Nómina, períodos de pago, cargos, AFP/ARS, tipos de ingreso/descuento                                                                      |
| comunicaciones   | Campañas y comunicados (correo/WhatsApp)                                                                                                   |
| seguridad        | Panel de seguridad de datos (cifrado, expiración de claves, etc.)                                                                          |
| auditoria        | Bitácora del sistema (quién hizo qué)                                                                                                      |
| automatizaciones | Tablero de alertas/automatizaciones                                                                                                        |
| entrenamiento    | Materiales/espacio de entrenamiento                                                                                                        |
| orientacion      | Orientación / historial clínico                                                                                                            |
| ia               | Utilidades complementarias (integración opcional)                                                                                          |

Los módulos **Caja, Facturación, Nómina, Asistencia, Comunicaciones** son

opcionales por centro: se muestran en el menú solo si el centro los tiene

habilitados y el rol tiene acceso.

## 1.3 Instalación local

```
git clone <repo>
cd mycolegiofavorito
python -m venv venv
```

```

venv\Scripts\activate          # Windows | source venv/bin/activate (Linux/macOS)
pip install -r requirements.txt
copy .env.example .env         # configurar SECRET_KEY y demás variables
python manage.py migrate
# Datos demo (opcional, incluye 1 superusuario admin/... y centro de ejemplo):
python manage.py seed_demo
python manage.py runserver

```

Variables relevantes de `.env` (ver `.env.example`):

| Variable                                                          | Uso                                                                   |
|-------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>SECRET_KEY</code>                                           | Clave secreta de Django (obligatoria)                                 |
| <code>DEBUG</code>                                                | <code>True</code> en desarrollo; <code>False</code> en producción     |
| <code>DB_ENGINE</code>                                            | <code>sqlite</code> (default) o <code>postgresql</code>               |
| <code>DATABASE_URL</code>                                         | Cadena Postgres de Render/Railway (prioridad sobre variables sueltas) |
| <code>REDIS_URL</code>                                            | Caché Redis (opcional)                                                |
| <code>ENCRYPTION_KEY</code>                                       | Clave Fernet para cifrar datos sensibles (fail-closed en producción)  |
| <code>ALLOWED_HOSTS</code> ,<br><code>CSRF_TRUSTED_ORIGINS</code> | Hosts permitidos                                                      |
| <code>EMAIL_*</code> , <code>WHATSAPP_*</code>                    | Correo y WhatsApp para el módulo de comunicaciones                    |

## 1.4 Comandos de gestión (management commands)

| Comando                          | Función                                                                                                                                                          |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>seed_inicial</code>        | Inicializa roles base y permisos por defecto ( <code>seed_permisos</code> ) y datos demo si el centro principal está vacío. Se ejecuta en el build de producción |
| <code>seed_permisos</code>       | Crea/actualiza <code>PermisoPagina</code> y <code>RolCentro</code> . Idempotente. <code>--solo-faltantes</code> no toca permisos existentes                      |
| <code>seed_demo</code>           | Carga datos demo del centro principal (docentes, estudiantes, tutores, administrativos, estructura MINERD, nómina, boletines, etc.)                              |
| <code>seed_planes_demo</code>    | Datos demo adicionales (planes, etc.)                                                                                                                            |
| <code>bootstrap_superuser</code> | Crea el superusuario inicial en producción                                                                                                                       |

**Usuarios demo ( `seed_demo` )**

| Usuario     | Contraseña    | Rol                                                            |
|-------------|---------------|----------------------------------------------------------------|
| admin       | admin123      | superusuario (superadmin) — TOTP <code>JBSWY3DPEHPK3PXP</code> |
| director    | admin123      | director                                                       |
| secretaria  | admin123      | secretaria                                                     |
| cajero      | admin123      | cajero                                                         |
| docente     | docente123    | docente (Matemática, Ciencias Sociales)                        |
| docente2    | docente123    | docente (Lengua Española, Inglés, Ed. Artística)               |
| docente3    | docente123    | docente (Ciencias de la Naturaleza, Ed. Física, Formación)     |
| estudiantes | estudiante123 | estudiante                                                     |

## 1.5 Despliegue en Render

El repo incluye `render.yaml` (Blueprint). Flujo: **dashboard.render.com** → **New** → **Blueprint** → **conectar este repo**.

```
buildCommand: pip install + collectstatic --noinput + migrate --noinput + seed_inicial .
startCommand: gunicorn mycolegiofavorito.wsgi:application.
```

Base de datos Postgres enlazada automáticamente ( `DATABASE_URL` ).

`ENCRYPTION_KEY`: Render no la genera válida; generarla con:

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

y pegarla en **Environment**. Sin ella el sistema NO arranca en producción.

Primer acceso: en la pestaña **Shell** del servicio ejecutar

```
DJANGO_SUPERUSER_PASSWORD='...' python manage.py bootstrap_superuser
```

Esa cuenta es el `superadmin` global.

## 1.6 Seguridad

**2FA (TOTP)**: al primer login con TOTP activo se pide el código de 6 dígitos; si el usuario no lo configuró aún, el sistema lo lleva a "Configurar 2FA".

**Permisos por página y rol**: cada URL tiene un registro `PermisoPagina` que define qué roles entran. Si una página no tiene registro, está abierta a todos los autenticados (ver `02-roles-y-permisos.md`).

**Cifrado de datos sensibles**: campos como cédula/teléfono se cifran con Fernet ( `ENCRYPTION_KEY` ) antes de guardar.

**Middlewares de defensa**: bloqueo de fuerza bruta (admin), cierre de sesión por inactividad, expiración de contraseñas, cabeceras de seguridad y CSP.

**Auditoría**: todo movimiento significativo se registra en la **Bitácora** (usuario, acción, módulo, IP, ruta).

## 1.7 Flujo de sesión

---

Login ( </login/> ).

Si el usuario pertenece a más de un centro → selección de centro.

`core:home` muestra el sidebar según el rol y los módulos activos del centro.

Cerrar sesión devuelve a login.

## 1.8 Regla de prueba del proyecto

---

```
python manage.py test
```

Enlaces a documentación técnica puntual en `md/` (promociones, correo): ver `00-indice.md`.